claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_56dfabe6-f8c\agent-a55718ffab8211afa.jsonl`
- SHA-256: `9e6708ad87b90ad28e2cc8520fe8ce607def6d97dc215902d748e415c9e1c4c6`
- Source modified: `2026-07-07T00:22:47+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_56dfabe6-f8c`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-07T00:20:30.164Z)

Reviewing GitHub PR #57 "F7: add /why explainability command" (branch feat/f7-why-command -> main). ADR 0012 F7 (final phase). +295/-2, 5 files.
Checked-out copy at C:/Users/avidu/Projects/_wt-pr57. Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr57/src C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F7 DOES:
- command_service.py: a /why command (regex allows "/why" alone or "/why <arg>"). _handle_why(conn, user_id, arg, cfg): if arg is None or
  not arg.strip().isdecimal() or int(arg) < 1 -> Usage reply. Else item = db.get_content_item_for_account(conn, user_id, item_id);
  if item is None -> "Content item `<id>` was not found for your account."; else decisions = db.list_policy_decisions_for_content(item.id);
  if decisions -> _format_policy_decisions(item, decisions); else -> _format_missing_policy_decision(item, current_cfg).
- db.py get_content_item_for_account: SELECT * FROM content_items WHERE id=? AND account_id=? (the account-scoped IDOR guard).
- _format_policy_decisions renders each decision + stage (reason + metadata via _format_json_value = json.dumps(default=str, ensure_ascii=False), each value truncated to 500 chars).
- _format_missing_policy_decision: replays the CURRENT FilterConfig via FilterEngine over a VideoMeta built from persisted content_item fields
  (duration/width/height/size/mime only), with a caveat that historical config changes may differ.

VERIFIED by the main reviewer: gate green (560 passed @ 93.99%); IDOR handled ? account A doing /why on account B item returns "not found for your account"
with NO leak of B channel/caption/url/domain; B sees own trace; malformed (/why, /why abc, /why -1, /why 0) -> Usage. Tests cover cross-account access + scoping.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR diff. No praise. Concrete failure scenario, ranked.
Severity: high (access-control bypass / cross-account leak, or crash on user input), medium (real edge bug), low (rendering/limit/note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr57/src C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. CONFIRMED only if it genuinely ships in this diff and produces the wrong behaviour. REFUTED if handled / cannot occur / acceptable-v1. PLAUSIBLE if unsettled. Corrected severity (cross-account leak/crash=high; edge=medium; note=low).
FINDING (parse_render): Markdown injection into the /why report via attacker-controlled caption / link_url
severity medium | src/tg_video_filter/command_service.py:204

summary: `item.text` (the caption of a watched channel's post, attacker-controlled) is echoed into the markdown-parsed /why reply at line 204, and `item.link_url` raw at line 202. Telethon sends these with markdown parsing enabled (send_message with no parse_mode, messages use `**bold**`/backticks). The JSON-encoding at line 176 escapes quotes/backslashes but does NOT neutralize markdown metacharacters ([ ] ( ) * _ `), so an attacker's caption can inject active formatting ? most notably a clickable [label](url) hyperlink ? into the bot's own trusted explainability output.
scenario: An attacker who controls a post in a channel the victim watches sets the caption to `Nice video [CLICK FOR PRIZE](http://phish.evil)`. That lands as item.text. When the victim runs `/why <id>`, line 204 renders `Text: "Nice video [CLICK FOR PRIZE](http://phish.evil)"`; Telethon's markdown parser turns it into a real MessageEntityTextUrl ? the phishing link appears as trusted-looking text inside the bot's report. link_url (line 202, currently often NULL) is even more exposed as it is rendered with no encoding at all.
evidence: item.text is populated from real message captions (_message_text at telethon_client.py:279 / backfill.py:243) and persisted (db.py:1813,1867), so it is live attacker data. Reproduced through telethon.extensions.markdown.parse on the actual `_format_json_value(caption)` output: the caption `[CLICK FOR PRIZE](http://phish.evil)` produced a MessageEntityTextUrl entity in the final message; `[pwn](http://x)` in a link_url likewise produced a MessageEntityTextUrl. JSON encoding left `[ ] ( )` intact.
rec: Escape markdown metacharacters in attacker-controlled values before interpolation (or send this reply with parse_mode disabled and format entities explicitly, or wrap echoed text as inline code with the backtick content itself escaped). Apply to item.text and item.link_url, and to any stage `reason` that can carry echoed content.

2. assistant (2026-07-07T00:20:34.222Z)

I'll verify this finding by reading the cited code and running a repro.

3. user (2026-07-07T00:20:35.051Z)

```
150                 "? **Content exclusions** (all off by default)",
151                 "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
152                 "`/set exclude round on|off` ? Exclude circular video messages",
153                 "`/set exclude nosound on|off` ? Exclude videos without audio\n"
154                 "   Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
too.",
155                 "",
156                 "? **Backfill**",
157                 "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
158                 "`/stop` ? Cancel running backfill",
159                 "",
160                 "? **Source scanning**",
161                 "`/channels` ? List all known sources with watch status",
162                 "`/watch <id|@name>` ? Start watching a source for new videos",
163                 "`/unwatch <id|@name>` ? Stop watching a source",
164                 "`/watchlist` ? Show only watched sources",
165                 "",
166                 "? **Explainability**",
167                 "`/why <content_item_id>` ? Explain why an item passed or failed",
168                 "",
169                 "\u2139\ufe0f  **Help**",
170                 "`/show commands` ? Show this help message",
171             ]
172         )
173
174
175     def _format_json_value(value: object, max_len: int = 500) -> str:
176         text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)
177         if len(text) <= max_len:
178             return text
179         return f"{text[: max_len - 1]}?"
180
181
182     def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
```

```
183         """Build a metadata replay input from fields persisted on content_items."""
184         return VideoMeta(
185             duration_s=item.duration_s,
186             width=item.width,
187             height=item.height,
188             size_bytes=item.size_bytes,
189             mime_type=item.mime_type,
190         )
191
192
193     def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
194         status = "PASSED" if decisions[-1].passed else "REJECTED"
195         lines = [
196             f"? **Why content item `{item.id}`?**",
197             "",
198             f"Final policy result: **{status}**",
199             f"Provider message: `{item.provider}:{item.provider_message_id}`",
200         ]
201         if item.link_url:
202             lines.append(f"Link: {item.link_url}")
203         if item.text:
204             lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
205
206         for decision in decisions:
207             decision_status = "PASSED" if decision.passed else "REJECTED"
208             lines.extend(
209                 [
210                     "",
211                     f"Policy `{decision.policy_id}`: **{decision_status}**",
212                     f"Evaluated: `{decision.evaluated_at.isoformat()}`",
213                 ]
214             )
215             if not decision.trace:
216                 lines.append("- No stage trace recorded.")
217                 continue
218             for stage in decision.trace:
219                 stage_name = str(stage.get("stage_name", "unknown"))
220                 stage_passed = bool(stage.get("passed"))
221                 reason = str(stage.get("reason", "no reason recorded"))
222                 icon = "?" if stage_passed else "?"
223                 lines.append(f"- {icon} `{stage_name}` ? {reason}")
224                 metadata = stage.get("metadata")
225                 if isinstance(metadata, dict) and metadata:
226                     lines.append(f"  metadata: `{_format_json_value(metadata)}`")
227         return "\n".join(lines)
228
229
```

4. assistant (2026-07-07T00:20:38.264Z)

Now let me look at how this reply is actually sent ? the parse_mode matters. Let me find the send path.

5. user (2026-07-07T00:20:40.302Z)

```
193:def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
230:def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) -> str:
429:            return await self._handle_why(conn, user_id, m.group(1), current_cfg)
501:    async def _handle_why(
522:            return CommandResult(_format_policy_decisions(item, decisions))
523:        return CommandResult(_format_missing_policy_decision(item, current_cfg))
```

6. user (2026-07-07T00:20:41.242Z)

```
264:    The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
268:    reply_text: str = ""
281:    transport (checking message.out, sending the reply); the Bot API adapter
290:        await send_reply(result.reply_text)
302:        """Parse and execute a command text, returning the reply + change flag."""
```

7. assistant (2026-07-07T00:20:43.722Z)

Now I need to find how `send_reply` / `reply_text` is actually sent ? specifically the parse_mode used by the Telethon adapter. This is the crux of the finding.

8. user (2026-07-07T00:20:45.665Z)

```
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\backfill.py:365:            await client.send_message("me", text)
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\command_service.py:114:    """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\command_service.py:264:    The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\command_service.py:268:    reply_text: str = ""
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\command_service.py:290:        await send_reply(result.reply_text)
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\commands.py:86:        if result.reply_text:
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\commands.py:87:            await _reply(client, message, result.reply_text)
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\commands.py:130:        await client.send_message("me", text)
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\delivery.py:177:    ok = await _send_with_retry(client, lambda: client.send_message(target, text), video_id)
C:\Users\avidu\Projects\_wt-pr57\src\tg_video_filter\models.py:362:    # The string is passed verbatim to Telethon's forward_messages / send_message.
```

9. user (2026-07-07T00:20:49.083Z)

```
70                  return
71
72              text: str = getattr(message, "message", "") or ""
73              client: TelegramClient = getattr(event, "client", None)  # type: ignore[assignment]
74
75              # Defer DB load ? skip non-command self-messages (review finding #2).
76              if not text.startswith("/"):
77                  return
78
79              current_cfg = await db.load_filter_config(conn, user.id)
80              result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
81
82              if result.config_changed:
83                  new_cfg = await db.load_filter_config(conn, user.id)
84                  engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
85
86              if result.reply_text:
87                  await _reply(client, message, result.reply_text)
88
89          return on_command
90
91
92      # --- backwards-compat helpers (test_commands.py references) ------------------
93
94
95      async def _update_config(
96          conn: aiosqlite.Connection,
```

```
97          user_id: int,
98          max_size_mb: int | None = None,
99          max_duration_s: int | None = None,
100         min_duration_s: int | None = None,
101         target_height: int | None = None,
102         exclude_gif: bool | None = None,
103         exclude_round: bool | None = None,
104         exclude_nosound: bool | None = None,
105     ) -> FilterConfig:
106         """Deprecated wrapper ? delegates to ``CommandService._update``.
107
108         Kept for backwards-compatibility with ``tests/test_commands.py`` which
109         imports ``_update_config`` from here. New code should call
110         :meth:`CommandService.handle` which returns a :class:`CommandResult`.
111         """
112         svc = CommandService()
113         cfg = await db.load_filter_config(conn, user_id)
114         return await svc._update(
115             conn,
116             user_id,
117             cfg,
118             max_size_mb=max_size_mb,
119             max_duration_s=max_duration_s,
120             min_duration_s=min_duration_s,
121             target_height=target_height,
122             exclude_gif=exclude_gif,
123             exclude_round=exclude_round,
124             exclude_nosound=exclude_nosound,
125         )
126
127
128     async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
129         try:
130             await client.send_message("me", text)
131         except (OSError, TimeoutError, RPCError):
132             log.warning("config_reply_failed", exc_info=True)
133
134
135     __all__ = [
136         "ENGINE_KEY",
137         "_EXCLUDE_RE",
138         "_LENGTH_RE",
139         "_MIN_LENGTH_RE",
```

10. assistant (2026-07-07T00:20:55.283Z)

The reply is sent via `client.send_message("me", text)` with no explicit `parse_mode`. In Telethon, the default parse_mode is markdown. Let me confirm whether this is in the PR diff or pre-existing, and check the Telethon default. Let me look at the diff and check the installed Telethon version.

11. user (2026-07-07T00:20:57.898Z)

```
12b4442 feat(commands): add why explainability command
6149c38 feat(policy): persist policy decision traces (#55)
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
---DIFF STAT---
 src/tg_video_filter/command_service.py | 120 +++++++++++++++++++++++++-
 src/tg_video_filter/commands.py        |   2 +
 src/tg_video_filter/db.py              |  16 ++++
 tests/test_command_service.py          | 146 +++++++++++++++++++++++++++++++-
 tests/test_db.py                       |  13 +++
 5 files changed, 295 insertions(+), 2 deletions(-)
```

12. user (2026-07-07T00:20:59.273Z)

```
1.44.0
parse_mode default attr exists
```

13. assistant (2026-07-07T00:21:02.086Z)

Telethon 1.44.0. Let me check its default parse_mode and confirm the diff for commands.py line 87/130 (whether `_reply` / send path is in the PR or pre-existing).

14. user (2026-07-07T00:21:03.258Z)

```
diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 3aea1f6..06db189 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -41,6 +41,7 @@ from tg_video_filter.command_service import (  # noqa: E402, F401
     _UNWATCH_RE,
     _WATCH_RE,
     _WATCHLIST_RE,
+    _WHY_RE,
     _format_duration,
     _format_size,
     format_commands,
@@ -142,6 +143,7 @@ __all__ = [
     "_EXCLUDE_HELP_RE",
     "_MIN_RES_HELP_RE",
     "_SIZE_RE",
+    "_WHY_RE",
     "_format_duration",
     "_format_size",
     "_update_config",
```

15. user (2026-07-07T00:21:05.737Z)

```
default parse_mode property:
This property is the default parse mode used when sending messages.
Defaults to
```

16. assistant (2026-07-07T00:21:09.947Z)

The `_reply`/`send_message("me", text)` path is pre-existing (not in this PR diff). The default parse_mode: let me confirm it's markdown, and run the actual injection repro through Telethon's markdown parser on the real formatted output.

17. user (2026-07-07T00:21:13.608Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 4, in <module>
    src = inspect.getsource(MessageMethods.parse_mode.fget)
                            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: type object 'MessageMethods' has no attribute 'parse_mode'
```

18. user (2026-07-07T00:21:19.525Z)

```
'self._parse_mode = markdown'
```

19. assistant (2026-07-07T00:21:22.780Z)

Confirmed: Telethon 1.44.0 defaults `_parse_mode = markdown`. Now the actual injection repro through the real code path ? build a ContentItem with an attacker caption, run `_format_policy_decisions`, and parse the output through Telethon's markdown parser.

20. user (2026-07-07T00:21:25.821Z)

```
ContentItem fields: ['id', 'account_id', 'source_id', 'provider', 'provider_message_id',
'content_type', 'text', 'link_url', 'media_ref', 'document_id', 'duration_s', 'width', 'height',
'size_bytes', 'mime_type', 'dedup_key', 'matched', 'seen_at']
```

21. user (2026-07-07T00:21:34.549Z)

```
ContentItem full: ['id=D', 'account_id', 'source_id', 'provider=D', 'provider_message_id',
'content_type=D', 'text=D', 'link_url=D', 'media_ref=D', 'document_id=D', 'duration_s=D',
'width=D', 'height=D', 'size_bytes=D', 'mime_type=D', 'dedup_key', 'matched=D', 'seen_at=D']

PolicyDecision: ['id=D', 'content_item_id', 'policy_id', 'passed', 'trace=D', 'evaluated_at=D']
```

22. user (2026-07-07T00:21:43.276Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 13, in <module>
    dec = PolicyDecision(content_item_id=42, policy_id='p1', passed=True,
                         trace=[], evaluated_at=datetime(2026,1,1,tzinfo=timezone.utc), id=1)
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for PolicyDecision
policy_id
  Input should be a valid integer, unable to parse string as an integer [type=int_parsing,
input_value='p1', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/int_parsing
```

23. user (2026-07-07T00:21:51.244Z)

```
=== RAW REPORT ===
? **Why content item `42`?**

Final policy result: **PASSED**
Provider message: `telethon:999`
Link: http://real[pwn](http://x.evil)
Text: "Nice video [CLICK FOR PRIZE](http://phish.evil)"

Policy `7`: **PASSED**
Evaluated: `2026-01-01T00:00:00+00:00`
- No stage trace recorded.

=== VISIBLE TEXT AFTER MARKDOWN PARSE ===
? Why content item 42?

Final policy result: PASSED
Provider message: telethon:999
Link: http://realpwn
Text: "Nice video CLICK FOR PRIZE"

Policy 7: PASSED
Evaluated: 2026-01-01T00:00:00+00:00
- No stage trace recorded.

=== ENTITIES ===
MessageEntityBold | url= None | offset 3 len 20 | covers= 'hy content item 42?\n'
MessageEntityCode | url= None | offset 20 len 2 | covers= '2?'
MessageEntityBold | url= None | offset 46 len 6 | covers= 'ASSED\n'
MessageEntityCode | url= None | offset 71 len 12 | covers= 'elethon:999\n'
MessageEntityTextUrl | url= http://x.evil | offset 101 len 3 | covers= 'wn\n'
MessageEntityTextUrl | url= http://phish.evil | offset 123 len 15 | covers= 'LICK FOR PRIZE"'
MessageEntityCode | url= None | offset 148 len 1 | covers= ':'
```

```
MessageEntityBold | url= None | offset 151 len 6 | covers= 'ASSED\n'
MessageEntityCode | url= None | offset 169 len 25 | covers= '026-01-01T00:00:00+00:00\n'
```

24. assistant (2026-07-07T00:21:58.068Z)

```
The injection is confirmed. Two `MessageEntityTextUrl` entities were produced:
- `url=http://phish.evil` covering "CLICK FOR PRIZE" (from `item.text` at line 204)
- `url=http://x.evil` covering "pwn" (from `item.link_url` at line 202)

Both are attacker-controlled captions/urls rendered as real clickable hyperlinks in the bot's
trusted /why output. The offsets are shifted by Telethon's known entity-offset quirk but the
entities and their URLs are real.

Now, the adversarial refutation checks:

1. **Is the vulnerable code in THIS PR diff?** Yes ? lines 202 and 204 (`Link: {item.link_url}`
and `Text: {_format_json_value(item.text...)}`) are new in `_format_policy_decisions`, added by
this PR (command_service.py +120, the whole `_format_policy_decisions` function is new).

2. **Is `item.text` attacker-controlled?** Let me verify the population claim and that markdown
is truly the send default (not overridden anywhere for this reply path).
```

25. user (2026-07-07T00:22:00.944Z)

```
62:+def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
70:+    if item.link_url:
71:+        lines.append(f"Link: {item.link_url}")
72:+    if item.text:
73:+        lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
166:+            return CommandResult(_format_policy_decisions(item, decisions))
```

26. user (2026-07-07T00:22:03.180Z)

```
async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
    try:
        await client.send_message("me", text)
    except (OSError, TimeoutError, RPCError):
        log.warning("config_reply_failed", exc_info=True)
```

27. assistant (2026-07-07T00:22:07.631Z)

```
Confirmed: lines 202/204 (`Link:`/`Text:`) are added by this PR, and `_reply` sends via
`send_message("me", text)` with no `parse_mode` override, so Telethon's markdown default
applies.

Now let me weigh the refutation more carefully ? the key question for severity. Does existing
bot output already echo attacker-controlled text through markdown (making this not-new), and is
`item.text` genuinely attacker-controlled? Let me check the population path and whether other
commands already do the same (to assess whether it's a regression vs. pre-existing pattern).
```

28. user (2026-07-07T00:22:10.063Z)

```
279:        text = _message_text(message)
393:def _message_text(message: object) -> str | None:
394:    """Return caption/message text normalized for storage, or None if empty."""
521:    "_message_text",
---backfill---
273:                    text=text,
315:                        link_text=link_text,
---db persist---
898:        text=row["text"],
1708:        text=row["text"],
1813:        text=video.text,
1867:        text=item.text,
```

```
1951:            text=r["text"],
```

29. assistant (2026-07-07T00:22:16.188Z)

`item.text` is populated from the raw message caption (`_message_text` normalizes and stores it) ? genuinely attacker-controlled data from a watched channel's post. Now the key refutation angle: does the existing codebase already echo attacker-controlled fields into markdown replies (e.g. `/channels` echoing a channel title)? If so, this is a pre-existing project pattern, not a regression introduced by this PR ? though the PR still adds new sinks. Let me check.

30. user (2026-07-07T00:22:19.236Z)

```
def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) -> str:
    recorded_status = "accepted" if item.matched else "rejected"
    replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
    replay_status = "accepted" if replay.matched else "rejected"
    lines = [
        f"? **Why content item `{item.id}`?**",
        "",
        "No policy-stage decision trace was recorded for this item.",
        f"Recorded item status: **{recorded_status}**.",
        "",
        "Current metadata-filter replay:",
        f"- Result with current config: **{replay_status}**",
    ]
    if replay.reasons:
        lines.extend(f"- {reason}" for reason in replay.reasons)
    else:
        lines.append("- No metadata filters are currently configured.")
    lines.extend(
        [
            "",
            "This fallback uses the current filter config and persisted metadata only; "
            "historical config changes can make it differ from the original decision.",
        ]
    )
    return "\n".join(lines)
```

31. assistant (2026-07-07T00:22:34.400Z)

The missing-decision path doesn't echo `item.text`/`link_url`, so the injection is specific to the `_format_policy_decisions` path (lines 202/204).

Now the honest severity assessment. This is a real, newly-introduced markdown injection: attacker-controlled caption/link_url rendered as clickable hyperlinks in the bot's trusted output. Refutation attempts all failed:
- Not handled (JSON encoding leaves `[](){}` intact ? verified)
- Genuinely attacker-controlled (`item.text` = channel caption)
- Genuinely ships in this diff (lines 202/204 are new)
- Markdown is genuinely the send default (Telethon 1.44.0, no override)

Severity calibration: This is not a cross-account leak (the IDOR guard is intact ? the victim only sees their own item's caption). It's not a crash. The attacker controls content in a channel the victim *already watches* and reads. The bot reply goes to the operator's own Saved Messages ("me"). The impact is a phishing hyperlink / formatting spoof injected into the bot's trusted self-report ? a real but bounded rendering/trust issue. Per the rubric this is squarely "note" territory (rendering/limit) leaning medium (it forges a clickable link the user may trust as bot-vetted). The finding claims medium; I'll confirm at medium ? it's a genuine injection with a plausible phishing vector, above pure cosmetic, but below access-control/crash.

32. user (2026-07-07T00:22:46.917Z)

Structured output provided successfully